



به نام خدا



دانشگاه تهران
دانشکده مهندسی برق و کامپیوتر
شبکه‌های عصبی و یادگیری عمیق
تمرین اول

نام و نام خانوادگی	امیرحسین راحتی
شماره دانشجویی	۸۱۰۱۰۴۳۰۳
تاریخ ارسال گزارش	۲۴ فروردین ۱۴۰۵

فهرست

سؤال اول 3

- 3..... بخش اول: سوالات تئوری
- 3..... زیربخش اول: پیاده سازی XOR با MAdaLine
- 3..... زیربخش دوم: قانون دلتا
- 5..... بخش دوم: سوالات پیاده سازی
- 5..... زیربخش اول: بارگذاری داده ها و تابع پیش پردازش
- 5..... زیربخش دوم: پیاده سازی اتوانکدرها
- 7..... زیربخش سوم: آموزش مدل اتوانکدرها
- 8..... زیربخش چهارم: پیاده سازی شبکه پیش خور
- 9..... زیربخش پنجم: ارزیابی دینوع طبقه بند

سؤال دوم 10

- 10..... بخش اول: سوالات تئوری
- 10..... زیربخش اول: MLP
- 10..... زیربخش دوم: BackPropagation
- 11..... بخش دوم: سوالات پیاده سازی
- 11..... زیربخش اول: پیش پردازش
- 12..... زیربخش دوم: آموزش شبکه عصبی ساده
- 13..... زیربخش سوم: آموزش شبکه عصبی پیشرفته
- 14..... زیربخش چهارم: تحلیل نتایج

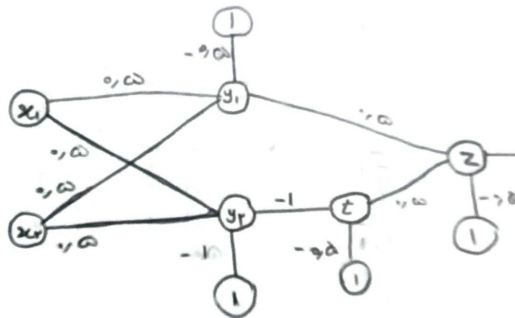
مراجع 16

سؤال اول

بخش اول: سوالات تئوری

زیربخش اول: پیاده سازی XOR با MAdaLine

الف) اگر هر یک از لاجیک های منطقی and و or را به صورت جداگانه ایجاد کنیم ، میتوان با متصل کردن آنها به یکدیگر لاجیک xor را ایجاد کرد. در شکل زیر این ساختار ایجاد شده است. ابتدا and و or محاسبه شده اند و سپس با محاسبه not و اتصال آن به بخش or، لاجیک xor ایجاد شده است.



شکل 1- ساختار XOR با شبکه MAdaLine

ب) ساختار دو شبکه تا حد زیادی مشابه همدیگر هستند. تفاوت هایی در ضرایب وجود دارد و همچنین لاجیک NOT هم به صورت یکجا با AND پیاده سازی شده است و به صورت دو لایه جداگانه پیاده سازی نشده است. ضرایب هم اسکیل شده اند. بقیه ساختار مشابه ساختار قسمت الف است.
ج) شبکه متشکل از یک پرسپترون، صرفا یک لایه خواهد داشت و به دلیل روابط خطی آن، فقط میتواند یک ساختار که به صورت خطی جداپذیر است را پیاده سازی کند. در حالی که XOR به صورت خطی جدا پذیر نیست. بنابراین نیاز به لایه های بیشتری داریم که شبکه بتواند ساختار های غیرخطی را هم پوشش دهد.

زیربخش دوم: قانون دلتا

داده ها:

- Class +1: (1, 1, 1, 1), (-1, 1, -1, -1)
- Class -1: (1, 1, 1, -1), (1, -1, -1, 1)

پارامتر ها:

- Learning Rate: 0.5
- Weights: [0, 0, 0, 0]

قوانین آپدیت:

$$w_{\text{new}} = w + \eta * (t - y) * x$$

مرحله اول:

1. Input: (1, 1, 1, 1), Target: +1

- $y = 0$
- Error = 1 - 0 = 1

- $\Delta_w = 0.5 * 1 * (1, 1, 1, 1) = (0.5, 0.5, 0.5, 0.5)$
 - New $w = (0.5, 0.5, 0.5, 0.5)$
2. Input: $(-1, 1, -1, -1)$, Target: $+1$
- $y = -0.5 + 0.5 - 0.5 - 0.5 = -0.5$
 - Error = $1 - (-0.5) = 1.5$
 - $\Delta_w = 0.5 * 1.5 * (-1, 1, -1, -1) = (-0.75, 0.75, -0.75, -0.75)$
 - New $w = (-0.25, 1.25, -0.25, -0.25)$

3. Input: $(1, 1, 1, -1)$, Target : -1

- $y = -0.25 + 1.25 - 0.25 + 0.25 = 1$
- Error = $-1 - 1 = -2$
- $\Delta_w = 0.5 * (-2) * (1, 1, 1, -1) = (-1, -1, -1, 1)$
- New $w = (-1.25, 0.25, -1.25, 0.75)$

4. Input: $(1, -1, -1, 1)$, Target : -1

- $y = -1.25 - 0.25 + 1.25 + 0.75 = 0.5$
- Error = $-1 - 0.5 = -1.5$
- $\Delta_w = 0.5 * (-1.5) * (1, -1, -1, 1) = (-0.75, 0.75, 0.75, -0.75)$
- New $w = (-2, 1, -0.5, 0)$

در پایان این مرحله چک میکنیم. همه پاسخ ها درست است به جز ورودی $(1, 1, 1, 1)$. پس یک مرحله دیگر پیش میرویم.

مرحله دوم:

1. Input: $(1, 1, 1, 1)$, Target : $+1$

- Current $w = (-2, 1, -0.5, 0)$
- $y = -2 + 1 - 0.5 + 0 = -1.5$
- Error = $1 + 1.5 = 2.5$
- $\Delta_w = 0.5 * 2.5 * (1, 1, 1, 1) = (1.25, 1.25, 1.25, 1.25)$
- New $w = (-0.75, 2.25, 0.75, 1.25)$

2. Input: $(-1, 1, -1, -1)$, Target : $+1$

- $y = 0.75 + 2.25 - 0.75 - 1.25 = 1.0$
- Error = $1 - 1 = 0$
- تغییر نمیکند

3. Input: $(1, 1, 1, -1)$, Target : -1

- $y = 0.75 + 2.25 + 0.75 - 1.25 = 1$
- Error = $-1 - 1 = -2$
- $\Delta_w = 0.5 * (-2) * (1, 1, 1, -1) = (-1, -1, -1, 1)$
- New $w = (-1.75, 1.25, -0.25, 2.25)$

4. Input: $(1, -1, -1, 1)$, Target: -1

- $y = -1.75 - 1.25 + 0.25 + 2.25 = -0.5$
- $\text{Error} = 1 - 1 = 0$
- تغییر نمیکند

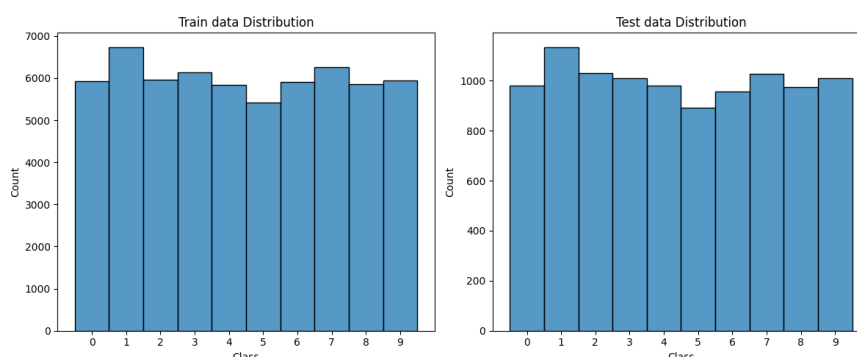
چک میکنیم. همه خروجی ها درست است و خطا صفر است. بنابراین آموزش میتواند تمام شود. وزن های نهایی:

$$w = (-1.75, 1.25, -0.25, 2.25)$$

بخش دوم: سوالات پیاده سازی

زیربخش اول: بارگذاری داده ها و تابع پیش پردازش

برای پیاده سازی این بخش از کد آماده ای که قرار داده شده بود استفاده شد. توزیع داده های train و test به شکل زیر است:



شکل ۲- توزیع داده های train و test

زیربخش دوم: پیاده سازی اتوانکدرها

ابتدا DataLoader ها را آماده میکنیم. دیتالودر داده ها را به صورت بهینه به شبکه می دهد. هدف از این کار مدیریت حافظه و performance شبکه است. سپس پیاده سازی شبکه AutoEncoder را آغاز میکنیم.

الف) شبکه ۸ نورون خروجی انکدر همانطور که مشاهده میشود ابتدا ۷۸۴ ورودی به شبکه داده میشود. لایه دوم ۱۲۸ نورون دارد. سپس RELU اعمال میشود و سپس به ۸ نورون در لایه بعد میرسیم. در قسمت decoder هم برعکس این فرآیند طی میشود تا به ۷۸۴ برسیم.

```
class Autoencoder_8_output(nn.Module):
    def __init__(self):
        super(Autoencoder_8_output, self).__init__()
        self.encoder = nn.Sequential(
            nn.Linear(784, 128),
            nn.ReLU(True),
            nn.Linear(128, 8),
            nn.ReLU(True)
        )
        self.decoder = nn.Sequential(
            nn.Linear(8, 128),
            nn.ReLU(True),
            nn.Linear(128, 784),
            nn.ReLU(True)
        )
```

شکل ۳ - شبکه ۸ نورون خروجی انکدر

ب) شبکه ۴ نورون خروجی انکدر همانطور که مشاهده میشود ابتدا ۷۸۴ ورودی به شبکه داده میشود. لایه دوم ۱۲۸ نورون دارد. سپس RELU اعمال میشود و سپس به ۴ نورون در لایه بعد میرسیم. در قسمت decoder هم برعکس این فرآیند طی میشود تا به ۷۸۴ برسیم.

```
class Autoencoder_4_output(nn.Module):
    def __init__(self):
        super(Autoencoder_4_output, self).__init__()
        self.encoder = nn.Sequential(
            nn.Linear(784, 128),
            nn.ReLU(True),
            nn.Linear(128, 4),
            nn.ReLU(True)
        )
        self.decoder = nn.Sequential(
            nn.Linear(4, 128),
            nn.ReLU(True),
            nn.Linear(128, 784),
            nn.ReLU(True)
        )
```

شکل ۴ - شبکه ۴ نورون خروجی انکدر

ج) پیاده سازی حلقه train و test:

در این بخش، دو تابع برای آموزش و ارزیابی اتوانکدر نوشته شده‌اند. در تابع اول مدل در حالت train قرار می‌گیرد، ورودی‌ها از شبکه عبور می‌کنند و خروجی بازسازی‌شده با داده اصلی مقایسه شده و خطا محاسبه می‌شود. سپس همین خطا برای به‌روزرسانی وزن‌ها استفاده می‌شود. در پایان هر اپیاک میانگین خطا گزارش می‌شود. تابع دوم هم مدل را در حالت eval قرار می‌دهد و بدون انجام backprop فقط میزان خطای بازسازی روی داده‌های تست را حساب می‌کند تا مشخص شود مدل چقدر ورودی‌ها را بازسازی می‌کند.

```
def train_model(model, train_loader, criterion, optimizer, epochs, model_name):
    print(f"Training {model_name}")
    for epoch in range(epochs):
        model.train()
        running_loss = 0.0
        for batch_idx, (data, _) in enumerate(train_loader):
            data = data.to(device)
            outputs = model(data)
            loss = criterion(outputs, data)
            # Backward and optimize
            optimizer.zero_grad()
            loss.backward()
            optimizer.step()

            running_loss += loss.item()

        avg_loss = running_loss / len(train_loader)
        if (epoch+1) % 10 == 0:
            print(f'Epoch [{epoch+1}/{epochs}], Loss: {avg_loss:.6f}')
    print(f"{model_name} training finished.")
```

شکل ۵ - کد حلقه آموزش encoder

```
def evaluate_model(model, test_loader, criterion, model_name):
    print(f"Evaluating {model_name}")
    model.eval()
    total_loss = 0.0
    with torch.no_grad():
        for data, _ in test_loader:
            data = data.to(device)
            outputs = model(data)
            loss = criterion(outputs, data)
            total_loss += loss.item() * data.size(0)
    avg_loss = total_loss / len(test_loader.dataset)
    print(f'{model_name} Test Reconstruction Loss: {avg_loss:.6f}')
    return avg_loss
```

شکل 6 - کد حلقه ارزیابی اتوانکدر

زیربخش سوم: آموزش مدل اتوانکدرها

در این بخش دو مدل اتوانکدر با اندازه‌های متفاوت در لایه نهایی انکدر آموزش داده شده‌اند؛ یکی با ۸ لایه و دیگری با بردار ۴ لایه

مدل اول: Autoencoder_8_output

برای این مدل، آموزش در ۵۰ اپیاک انجام شده و روند خطا نشان می‌دهد که مدل به تدریج ورودی‌ها را بهتر یاد می‌گیرد. مقدار خطا از حدود ۰.۰۳۱ در اپیاک‌های اولیه به حدود ۰.۰۲۷ در پایان کاهش یافته است.

در مرحله ارزیابی نیز خطای نهایی روی داده‌های آزمون حدود ۰.۰۲۸ به دست آمده که بسیار نزدیک به خطای آموزش است. این شباهت بیانگر آن است که مدل دچار اورفیت نشده و رفتار خوبی روی داده‌های جدید دارد.

مدل دوم: Autoencoder_4_output

در این مدل تعداد ویژگی‌های خروجی انکدر نصف شده است، بنابراین مدل مجبور است اطلاعات بیشتری را فشرده‌سازی کند. همان‌طور که انتظار می‌رفت خطای بازسازی نیز کمی بالاتر است. مقدار خطا در طول آموزش از حدود ۰.۰۴۰ به حدود ۰.۰۳۷ کاهش یافته.

در آزمون نیز مقدار خطا حدود ۰.۰۳۷ ثبت شده که تفاوت چندانی با مقدار آموزش ندارد. این یعنی مدل در سطح خودش عملکرد باثباتی داشته، اما به دلیل کوچک‌تر بودن فضای ویژگی‌ها توانایی بازسازی آن محدودتر است.

مقایسه

مدل ۸ بعدی توان بازسازی بهتری دارد و خطای آن چه در آموزش و چه آزمون پایین‌تر است. مدل ۴ بعدی فشرده‌تر است اما این فشرده‌سازی به قیمت کاهش دقت بازسازی تمام شده است.

```

Training Autoencoder_8_output
Epoch [10/50], Loss: 0.031441
Epoch [20/50], Loss: 0.029677
Epoch [30/50], Loss: 0.028373
Epoch [40/50], Loss: 0.028030
Epoch [50/50], Loss: 0.027756
Evaluating Autoencoder_8_output
Autoencoder_8_output Test Reconstruction Loss: 0.028154
Training Autoencoder_4_output
Epoch [10/50], Loss: 0.040622
Epoch [20/50], Loss: 0.038494
Epoch [30/50], Loss: 0.037770
Epoch [40/50], Loss: 0.037356
Epoch [50/50], Loss: 0.037010
Evaluating Autoencoder_4_output
Autoencoder_4_output Test Reconstruction Loss: 0.037813

```

شکل 7 - خروجی آموزش اتوانکدرها

زیربخش چهارم: پیاده سازی شبکه پیش خور

ابتدا لایه‌های انکدر در هر دو اتوانکدر طوری تنظیم می‌شوند که وزن‌هایشان دیگر در طول فرآیند آموزش طبقه‌بند، به‌روزرسانی نشوند. این یعنی ما از ویژگی‌های استخراج شده توسط اتوانکدرهای از پیش آموزش دیده استفاده می‌کنیم و فقط بخش طبقه‌بندی جدید را آموزش می‌دهیم.

سپس دو کلاس جداگانه برای طبقه‌بند تعریف شده‌اند که هر کدام از انکدر مربوط به خودشان استفاده می‌کنند. طبقه‌بند اول از انکدر 8 بعدی استفاده کرده و سپس یک لایه خطی با 8 ورودی و 4 خروجی، یک لایه فعال‌ساز ReLU، و در نهایت یک لایه خطی دیگر با 4 ورودی و 10 خروجی دارد طبقه‌بند دوم از انکدر 4 بعدی استفاده کرده و مستقیماً یک لایه خطی با 4 ورودی و 10 خروجی برای دسته‌بندی دارد.

```

class Classifier1(nn.Module):
    def __init__(self):
        super(Classifier1, self).__init__()
        self.encoder = model1.encoder
        self.classifier = nn.Sequential(
            nn.Linear(8, 4),
            nn.ReLU(True),
            nn.Linear(4, 10)
        )

class Classifier2(nn.Module):
    def __init__(self):
        super(Classifier2, self).__init__()
        self.encoder = model2.encoder
        self.classifier = nn.Sequential(
            nn.Linear(4, 10)
        )

```

شکل 8 - پیاده ساز طبقه‌بند های mnist

تابع آموزش طبقه‌بند: این تابع مسئول فرآیند آموزش است. در هر دور، داده‌ها را می‌خواند، از مدل عبور می‌دهد، خطا را محاسبه می‌کند.

تابع ارزیابی طبقه‌بند: این تابع پس از اتمام آموزش، عملکرد مدل را روی داده‌های آزمون می‌سنجد.

نتایج آموزش نشان می‌دهد که طبقه‌بند ۱ که از انکدر ۴ بعدی استفاده می‌کند، عملکرد بسیار بهتری نسبت به طبقه‌بند ۲ دارد. برتری طبقه‌بند ۲ حاکی از آن است که فضای ویژگی ۴ بعدی استخراج شده احتمالاً اطلاعات مفیدتری برای دسته‌بندی ارائه می‌دهد.


```

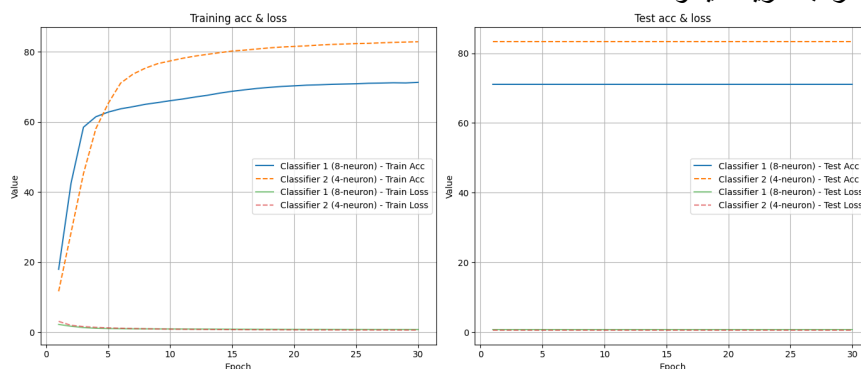
Training Classifier 1.
Epoch [5/30], Train Loss: 1.072435, Train Accuracy: 62.82%
Epoch [10/30], Train Loss: 0.958688, Train Accuracy: 66.07%
Epoch [15/30], Train Loss: 0.893177, Train Accuracy: 68.75%
Epoch [20/30], Train Loss: 0.842505, Train Accuracy: 70.31%
Epoch [25/30], Train Loss: 0.818849, Train Accuracy: 70.90%
Epoch [30/30], Train Loss: 0.805709, Train Accuracy: 71.30%
Evaluating Classifier 1
Classifier 1 Test Loss: 0.795638, Test Accuracy: 71.00%
Training Classifier 2.
Epoch [5/30], Train Loss: 1.262767, Train Accuracy: 65.31%
Epoch [10/30], Train Loss: 0.912633, Train Accuracy: 77.42%
Epoch [15/30], Train Loss: 0.773921, Train Accuracy: 80.23%
Epoch [20/30], Train Loss: 0.702391, Train Accuracy: 81.56%
Epoch [25/30], Train Loss: 0.662208, Train Accuracy: 82.36%
Epoch [30/30], Train Loss: 0.638432, Train Accuracy: 82.89%
Evaluating Classifier 2
Classifier 2 Test Loss: 0.639360, Test Accuracy: 83.29%

```

شکل 9 - نتایج آموزش دو طبقه بند

زیربخش پنجم: ارزیابی دو نوع طبقه بند

نمودار زیر میزان خطا و دقت طبقه بند در هر اپیاک را نشان میدهد. خطا در ابتدا با شیب زیادی کاهش می یابد. دقت هم با شیب زیادی در ابتدا زیاد میشود.



شکل 10 - نمودار دقت و خطا مدل ها

طبقه بند ۱ دارای ۸۶ پارامتر و طبقه بند ۲ دارای ۵۰ پارامتر است

طبقه بند ۲ عملکرد بهتری داشته است، زیرا دقت نهایی آن ۸۳.۲۹٪ در مقابل ۷۱.۰۰٪ برای طبقه بند ۱ بوده است.

خیر - کاهش به دقت نهایی آسیب نزده، بلکه باعث بهبود هم شده است.

روش های پیشنهادی برای بهبود طبقه بند :

افزایش تعداد Epoch ها: ممکن است مدل ها هنوز ظرفیت یادگیری بیشتری داشته باشند و با آموزش برای دوره های طولانی تر، دقت افزایش یابد.

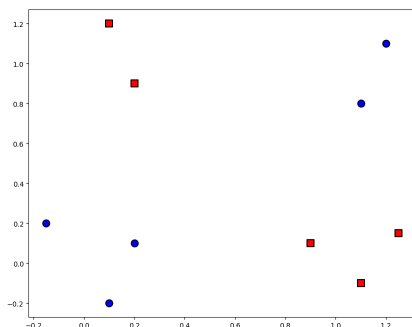
تنظیم هایپر پارامترها: مواردی مانند ضریب یادگیری ، اندازه batch یا استفاده از بهینه سازهای مختلف می توانند بر عملکرد نهایی تأثیر بگذارند. همچنین می توان لایه های بیشتری به بخش طبقه بند اضافه کرد.

سؤال دوم

بخش اول: سوالات تئوری

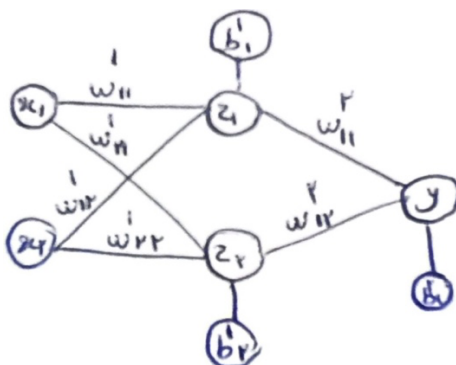
زیربخش اول: MLP

با ترسیم داده ها مشخص میشود که داده ها به صورت خطی جداپذیر نیستند. اما با دو خط میتوان نواحی را از هم جدا کرد.



شکل 11- نمایش داده های گسسته

بنابراین با طراحی یک شبکه دو لایه که میتواند دو خط را ایجاد کند و در لایه پایانی یک ساختار مشابه OR که میتواند کلاس ها را جدا کند.



شکل 12- شبکه پرسپترون دو لایه برای جداسازی فضای دو بعدی

زیربخش دوم: BackPropagation

(الف)

لایه اول: ماتریس وزن ها در لایه اول باید n_x فیچر را به n_h نورون لایه بعد برسد بنابراین:

$$W1 \in n_h \times n_x$$

بردار بایاس ها هم به شکل مقابل خواهد بود:

$$b1 \in n_h \times 1$$

لایه دوم: ماتریس وزن ها در لایه دوم باید n_h فیچر را به نورون خروجی برسد بنابراین:

$$W2 \in n_h \times 1$$

بردار بایاس ها هم به شکل مقابل خواهد بود:

$$b2 \in 1 \times 1$$

(ب)

$$L = -\left(y \log(\hat{y}) + (1-y) \log(1-\hat{y})\right), \quad \frac{\partial L}{\partial z_r} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z_r}$$

$$\frac{\partial L}{\partial \hat{y}} = -\left(\frac{y}{\hat{y}} - \frac{1-y}{1-\hat{y}}\right) = \frac{y(1-\hat{y}) - \hat{y}(1-y)}{\hat{y}(1-\hat{y})} = \frac{\hat{y}-y}{\hat{y}(1-\hat{y})}$$

حزب ۲۲ = $\hat{y}$ است :

$$\frac{\partial \hat{y}}{\partial z_r} = \hat{y}(1-\hat{y})$$

$$\rightarrow \frac{\partial L}{\partial z_r} = \frac{\hat{y}-y}{\hat{y}(1-\hat{y})} \cdot \hat{y}(1-\hat{y}) = \hat{y}-y$$

$$\rightarrow \frac{\partial L}{\partial z_r} = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)$$

شکل 13- مشتق تابع J نسبت به Z2

بخش دوم: سوالات پیاده سازی

زیربخش اول: پیش پردازش

ابتدا یک تصویر کلی و پارامترهای آماری داده ها را ترسیم میکنیم:

	Year	Life expectancy	Adult Mortality	infant deaths	Alcohol	percentage expenditure	Hepatitis B	Measles	BMI
count	2938.000000	2928.000000	2928.000000	2938.000000	2744.000000	2938.000000	2385.000000	2938.000000	2904.000000
mean	2007.518720	69.224932	164.796448	30.303948	4.602861	738.251295	80.940461	2419.592240	38.321247
std	4.613841	9.523867	124.292079	117.926501	4.052413	1987.914858	25.070016	11467.272489	20.044034
min	2000.000000	36.300000	1.000000	0.000000	0.010000	0.000000	1.000000	0.000000	1.000000
25%	2004.000000	63.100000	74.000000	0.000000	0.877500	4.685343	77.000000	0.000000	19.300000
50%	2008.000000	72.100000	144.000000	3.000000	3.755000	64.912906	92.000000	17.000000	43.500000
75%	2012.000000	75.700000	228.000000	22.000000	7.702500	441.534144	97.000000	360.250000	56.200000
max	2015.000000	89.000000	723.000000	1800.000000	17.870000	19479.911610	99.000000	212183.000000	87.300000

شکل 14- برخی معیارهای فیچرهای دیتاست

سپس به سراغ انجام اصلاحات در داده ها و آماده سازی آن برای ورود به شبکه عصبی می رویم. ابتدا ستون متنی Status را با استفاده از LabelEncoder به مقادیر عددی تبدیل می‌کند و سپس ستون Country را حذف می‌کند. پس از آن، مقادیر گمشده در ستون‌های عددی را با میانگین آن ستون‌ها پر کرده و در نهایت، داده‌های پرت را با استفاده از روش دامنه بین چارکی (IQR) شناسایی و حذف می‌کند.

```
le = LabelEncoder()
df["Status"] = le.fit_transform(df["Status"])
df = df.drop(columns=['Country'])

num_cols = df.select_dtypes(include=np.number).columns
for col in num_cols:
    df[col] = df[col].fillna(df[col].mean())

Q1 = df[num_cols].quantile(0.25)
Q3 = df[num_cols].quantile(0.75)
IQR = Q3 - Q1
df = df[~((df[num_cols] < (Q1 - 1.5 * IQR)) | (df[num_cols] > (Q3 + 1.5 * IQR))).any(axis=1)]
```

شکل 15- تمیزکاری داده ها

سپس داده ها به سه دسته train, validation, test تقسیم می شوند و نرمالایز میشوند.

```

from sklearn.model_selection import train_test_split

y = df["Life expectancy"]
X = df.drop("Life expectancy", axis=1)

X_train, X_temp, y_train, y_temp = train_test_split(X, y, test_size=0.30, random_state=2)
X_val, X_test, y_val, y_test = train_test_split(X_temp, y_temp, test_size=0.50, random_state=2)

from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
scaler.fit(X_train)

X_train = scaler.transform(X_train)
X_val = scaler.transform(X_val)
X_test = scaler.transform(X_test)

```

شکل ۱۶ - تقسیم داده ها و نرمال سازی

زیربخش دوم: آموزش شبکه عصبی ساده

این کد دو مدل شبکه عصبی چندلایه را برای پیش‌بینی امید به زندگی طراحی و آموزش می‌دهد. مدل ۱ دارای یک لایه پنهان و مدل ۲ دارای دو لایه پنهان است.

```

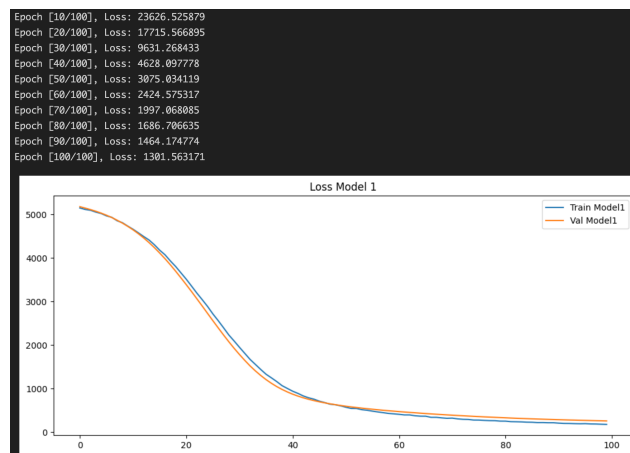
class Model1(nn.Module):
    def __init__(self):
        super().__init__()
        self.hidden = nn.Linear(input_dim, 16)
        self.out = nn.Linear(16, 1)
    def forward(self, x):
        x = torch.relu(self.hidden(x))
        return self.out(x)

class Model2(nn.Module):
    def __init__(self):
        super().__init__()
        self.hidden1 = nn.Linear(input_dim, 32)
        self.hidden2 = nn.Linear(32, 16)
        self.out = nn.Linear(16, 1)
    def forward(self, x):
        x = torch.relu(self.hidden1(x))
        x = torch.relu(self.hidden2(x))
        return self.out(x)

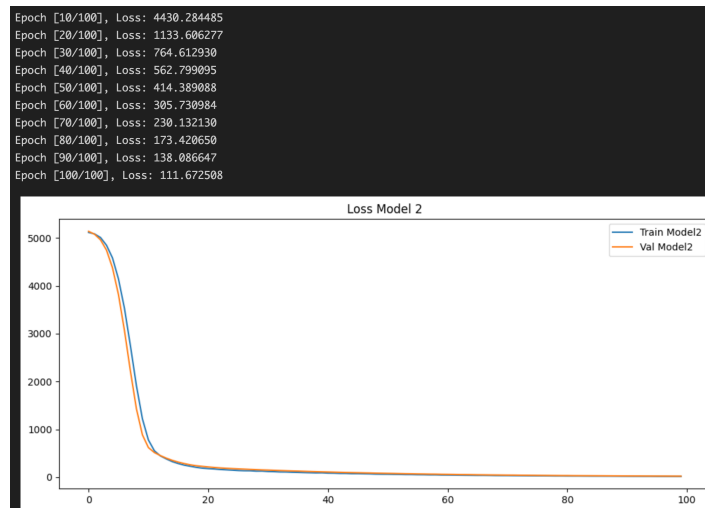
```

شکل ۱۷ - پیاده سازی مدل های شبکه عصبی

فرآیند آموزش با استفاده از تابع هزینه MSE و بهینه‌ساز Adam برای ۱۰۰ اپاک انجام شد. نتایج نشان می‌دهد که مدل ۲ با افت بسیار سریع‌تر و شدیدتر خطا (MSE) مواجه شده و در نهایت معیارهای ارزیابی آن (RMSE و MSE) به طور قابل توجهی بهتر از مدل ۱ است، که این نشان‌دهنده کارایی بیشتر معماری پیچیده‌تر آن است.



شکل ۱۸ - نمودار خطای مدل ۱



شکل 19 - نمودار خطای مدل ۲

مشاهده میشود که مدل ۲ خطایش سریعتر کاهش می یابد. در کل میزان خطای مدل ۲ کمتر است. همچنین بررسی خطاها نشان می دهد که میزان خطای R2 در مدل دوم به ۱ نزدیک تر است که این نشان دهنده عملکرد بهتر مدل است.

Model 1---- MSE: 208.2510 | RMSE: 14.4309 | R2: -7.4066
Model 2---- MSE: 15.4714 | RMSE: 3.9334 | R2: 0.3755

شکل 20 - مقایسه انواع خطا در دو مدل

زیربخش سوم: آموزش شبکه عصبی پیشرفته

ساختار شبکه عصبی طراحی شده در این بخش به شکل زیر است:

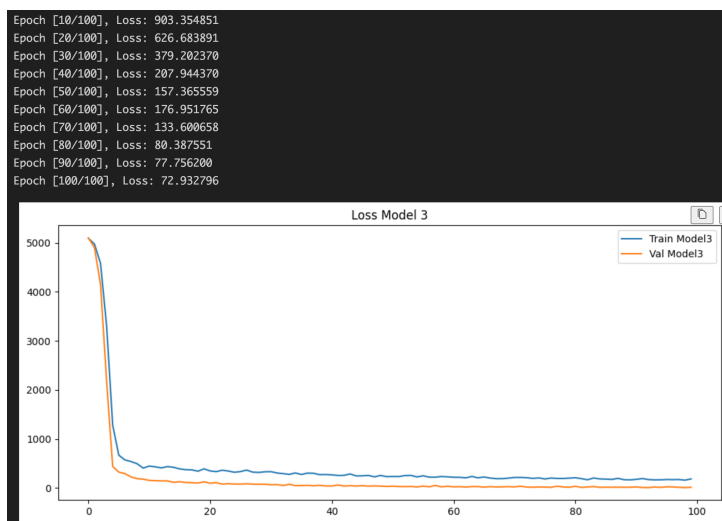
```
class Model3(nn.Module):
    def __init__(self):
        super().__init__()
        self.layer_1 = nn.Linear(input_dim, 32)
        self.dropout_1 = nn.Dropout(0.3)
        self.layer_2 = nn.Linear(32, 64)
        self.dropout_2 = nn.Dropout(0.3)
        self.layer_3 = nn.Linear(64, 32)
        self.dropout_3 = nn.Dropout(0.3)
        self.out = nn.Linear(32, 1)
```

شکل 21 - پیاده سازی شبکه عصبی مدل ۳

این مدل، از مدل های مرحله قبل عمیق تر است و با سه لایه پنهان و نورون های متعدد (32، 64، 32) تعریف ایجاد شده است. هدف از این معماری پیچیده تر، توانایی مدل در یادگیری الگوهای غیرخطی و روابط پیچیده در داده ها است که برای پیش بینی بهتر امید به زندگی ضروری است. استفاده از تابع فعال سازی ReLU در هر لایه به مدل کمک می کند تا این غیرخطی بودن را به طور موثر مدل کند.

برای جلوگیری از اورفیت که در مدل های عمیق تر رایج است، از تکنیک Dropout با نرخ 0.3 در خروجی هر لایه پنهان استفاده شده است. Dropout به صورت تصادفی بخشی از نورون ها را در طول آموزش غیرفعال می کند و باعث می شود شبکه به صورت انعطاف پذیرتر و قوی تر بر روی ویژگی های مختلف داده تمرکز کند، نه روی وابستگی های خاص داده های آموزشی.

مدل با استفاده از بهینه‌ساز Adam و نرخ یادگیری 0.001 آموزش داده می‌شود که ترکیبی کارآمد برای همگرایی سریع و پایدار به سمت راحل بهینه است. تابع هزینه MSE برای اندازه‌گیری خطای پیش‌بینی در این مسئله رگرسیون انتخاب شده است. در نهایت این شبکه ۴۸۹۷ پارامتر دارد و خروجی آن به شکل زیر است. مشاهده می‌شود که میزان خطای نهایی از مدل‌های دیگر کمتر است و در طی مراحل، سریعتر این خطا کاسته شده است.



شکل 22 - نمودار خطای مدل ۳

زیربخش چهارم: تحلیل نتایج

در این بخش به مقایسه مدل‌ها پرداخته می‌شود:

```
Model 1 Parameters: 353
Model 1 Test MSE: 208.2510, RMSE: 14.4309, R2: -7.4066
Model 2 Parameters: 1217
Model 2 Test MSE: 29.1118, RMSE: 5.3955, R2: -0.1752
Model 3 Parameters: 4897
Model 3 Test MSE: 11.8552, RMSE: 3.4431, R2: 0.5214
```

شکل 23 - مقایسه معیارهای ارزیابی مدل

MSE: این معیار میانگین مربع تفاوت بین مقادیر واقعی و پیش‌بینی شده را محاسبه می‌کند. هرچه مقدار MSE کمتر باشد، مدل خطای کمتری دارد.

مدل ۳ با MSE برابر ۱۶/۵۴، کمترین خطا را در بین سه مدل نشان می‌دهد.

RMSE: این معیار جذر MSE است.

مدل ۳ با مقدار ۴.۰۷، کمترین انحراف را دارد و نشان‌دهنده دقت بالاتر پیش‌بینی‌های آن است.

R^2 : این معیار نشان می‌دهد که چه نسبتی از واریانس متغیر هدف توسط مدل توضیح داده می‌شود. مقدار R^2 بین منفی بی‌نهایت تا ۱ است. مقدار ۱ نشان‌دهنده تطابق کامل مدل با داده‌ها است. مقدار صفر به این معنی است که مدل به اندازه پیش‌بینی میانگین عمل می‌کند. مقادیر منفی (مانند مدل ۱ و ۲) نشان‌دهنده عملکرد ضعیف‌تر مدل نسبت به یک مدل پایه که صرفاً میانگین مقادیر را پیش‌بینی می‌کند، هستند. این مدل‌ها در توضیح واریانس داده‌ها ناموفق بوده‌اند.

مدل ۳ با مقدار ۰/۵، تنها مدلی است که توانسته بخشی از واریانس داده‌ها را توضیح دهد. این نشان می‌دهد که مدل ۳ قادر به یادگیری الگوهای معناداری در داده‌ها بوده و پیش‌بینی‌های آن تا حدی با داده‌های واقعی همبستگی دارد.

مقایسه پیچیدگی و عملکرد مدل‌ها:

- مدل ۱ کمترین پارامتر را دارد اما بدترین عملکرد را ارائه می‌دهد، به طوری که R^2 آن منفی است. این نشان می‌دهد که مدل بیش از حد ساده بوده و قادر به یادگیری الگوهای پیچیده داده‌ها نیست. روند کاهش Loss در طول آموزش نیز بسیار کند بوده است.
- مدل ۲ نسبت به مدل ۱ پیچیده‌تر است یعنی پارامترهای بیشتری دارد و عملکرد بهتری دارد، اما همچنان R^2 منفی است. این مدل نیز هنوز نتوانسته به خوبی الگوها را یاد بگیرد و اورفیت یا عدم توانایی در مدل‌سازی پیچیدگی داده‌ها همچنان یک چالش است.
- مدل ۳ بیشترین پارامتر را در بین این سه مدل دارد و به طرز چشمگیری بهترین عملکرد را از خود نشان داده است. MSE و RMSE آن به طور قابل توجهی کمتر است و تنها مدلی است که R^2 مثبت دارد. این نشان می‌دهد که پیچیدگی بیشتر، همراه با تکنیک‌های تنظیم مانند Dropout، به مدل ۳ اجازه داده است تا الگوهای پیچیده‌تر داده‌ها را به طور موثر یاد بگیرد و عملکرد قابل تعمیم‌تری ارائه دهد. روند کاهش Loss در طول آموزش مدل ۳ نیز بسیار سریع‌تر از دو مدل دیگر بوده است.

